In [2]:
```
from pdf2image import convert_from_path
from IPython.display import display
images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W10
for img in images:
    display(img)
```

Physics 336
HW10

Do two of these in Mathematica and the other two in Python.

1. Suppose, for an unknown relationship, $y(x)$, we have the data points

$$x = \{1.0, 2.5, 3.5, 7.0, 11.2, 15.1\}$$
$$y = \{2.6, 4.8, 5.1, 4.9, 9.6, 12.3\}$$

   (a) Using linear interpolation, determine $y(3.0)$ and $y(9.0)$.

   (b) Construct a cubic spline fit and determine $y(3.0)$ and $y(9.0)$. Compare to your result from (a)

2. A weak x-ray emission peak is superimposed on top of a Bremsstrahlung background. The intensity of emmision is measured as a function of wavelength:

$$\lambda(\text{Å}) = \{2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 5.0, 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6.0\}$$
$$I(\text{arbitrary units}) = \{0.00, 0.89, 1.65, 2.31, 2.91, 3.52, 4.18, 4.92, 5.60, 6.02, 6.09, 5.91, 5.70, 5.59, 5.58, 5.64, 5.71, 5.78, 5.84, 5.89, 5.93, 5.95, 5.98, 5.99, 6.00, 6.01, 6.00, 5.99, 5.98, 5.97, 5.95, 5.93, 5.91, 5.88, 5.86, 5.83\}$$

   (a) Plot the data, and from the plot estimate the wavelength of the emission peak.

   (b) Create a spline fit for the background.

   (c) Subtract the background to isolate the emission peak.

      i. Fit this peak to a Gaussian, and estimate the peak.

      ii. Compare this peak to your estimate from the original data plot.

3. The position of a cart sliding down an air track is recorded as a function of time.

$$t(\text{s}) = \{0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7, 1.8, 1.9, 2.0, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 5.\}$$
$$x(\text{cm}) = \{6.29, 6.52, 6.58, 6.79, 6.9, 6.89, 7.05, 7.00, 7.18, 7.36, 7.58, 7.72, 7.78, 7.95, 7.91, 8.18, 8.34, 8.57, 8.67, 8.78, 9.07, 9.06, 9.48, 9.58, 9.63, 9.9, 10.23, 10.32, 10.44, 10.82, 10.95, 11.2, 11.39, 11.63, 11.98, 12.15, 12.49, 12.62, 12.93, 13.22, 13.36, 13.57, 13.85, 14.26, 14.42, 14.68, 15.13, 15.48, 15.6, 15.9, 16.22\}$$

   (a) Numerically differentiate this data to find and plot $v(t)$.

      i. Fit this to a line.

ii. Try smoothing the data by averaging over 3 points or 5 points. Fit the smoothed graph to a line and compare. Do this by smoothing the orignal data, and then by smoothing the differentiated data.

(b) Calculate the second derivative to find $a(t)$.

i. Plot your results and comment.

ii. Calculate the mean and standard deviation.

iii. Does it matter whether you use the original unsmoothed data, or the smoothed data?

(c) Fit the original data to a quadratic.

i. Differentiate this fit function to find $v(t)$, and compare to your previous results.

ii. Differentiate to find $a(t)$, and compare to your previous results.

4. A spring has one end fixed, while the other end is pulled with a variable force, F. The amount the spring stretches, x, is measured:

$$x(\text{cm}) = \{5.0, 10.0, 15.0, 20.0, 25.0, 30.0, 35.0, 40.0\}$$
$$F(\text{N}) = \{1.9, 3.7, 5.6, 7.3, 9.2, 11.0, 12.9, 14.6\}$$

(a) Construct a linear fit to find the spring constant, k.

(b) Integrate $\int F dx$, using data points:

i. Use a rectangular approximation.

ii. Use the trapezoidal approximation.

(c) Compare your results to $\frac{1}{2}kx^2$.

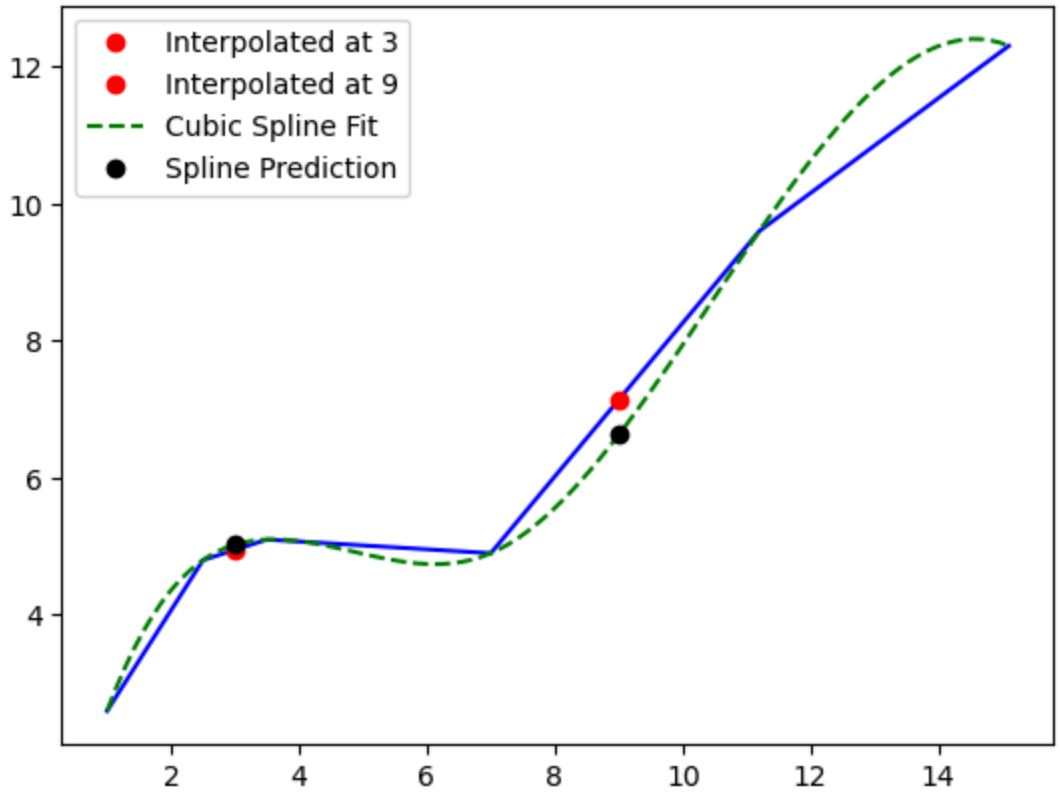
(d) Integrate your fit function, and compare to your results above.

Problem 1.

In [27]:
```
from matplotlib import pyplot as plt
import numpy as np
from scipy.interpolate import CubicSpline

x=[1.0, 2.5, 3.5, 7.0, 11.2, 15.1]
y=[2.6, 4.8, 5.1, 4.9, 9.6, 12.3]
```

```
##### Part a, linear interpolation #####
def linear(x_in):
    for i, x_val in enumerate(x):
        if x_val < x_in:
            idx = i
        else:
            break
    x1,x2 = x[idx], x[idx+1]
    y1,y2 = y[idx], y[idx+1]
    return y1 + (y2-y1)/(x2-x1)*(x_in-x1)

points=[3,9]

plt.plot(x,y, color = "blue")
for i in points:
    y_out = linear(i)
    plt.plot(i, y_out,'ro', label=f"Interpolated at {i}")
    print(f"At x={i}, interpolated y={y_out:.2f}")


##### Part b, cubic interpolation #####

CS = CubicSpline(x,y)
y_spline = CS(points)

x_smooth = np.linspace(min(x), max(x), 100)
y_smooth = CS(x_smooth)

plt.plot(x_smooth, y_smooth, label = "Cubic Spline Fit", color="green", linestyle="--")
plt.plot(points, y_spline, 'ko', label = "Spline Prediction")

for i, val in enumerate(points):
    print(f"Cubic Spline at x={val}: {y_spline[i]:.4f}")

plt.legend()
plt.show()
```

```
At x=3, interpolated y=4.95
At x=9, interpolated y=7.14
Cubic Spline at x=3: 5.0262
Cubic Spline at x=9: 6.6277
```



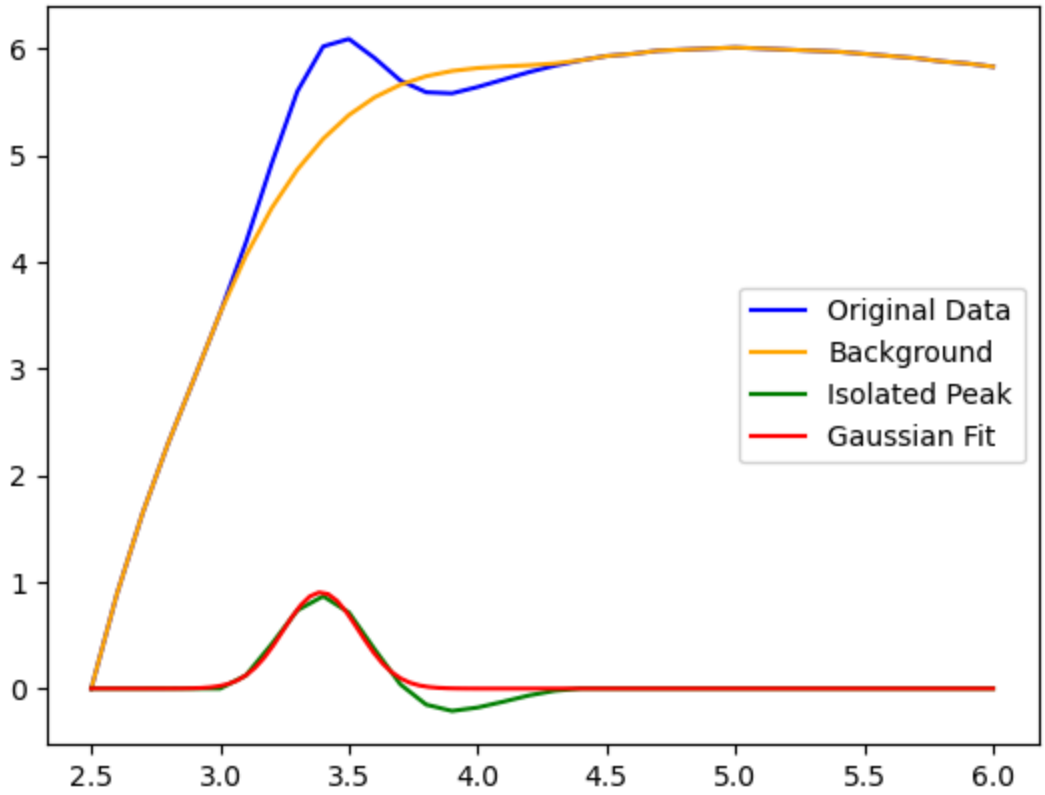
Problem 2

In [102]:

```
from matplotlib import pyplot as plt
import numpy as np
from scipy.interpolate import CubicSpline
from scipy.optimize import curve_fit

lam = [2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3, 3.4, 3.5, 3.6,
       3.7, 3.8, 3.9, 4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8,
       4.9, 5.0, 5.1, 5.2, 5.3, 5.4, 5.5, 5.6, 5.7, 5.8, 5.9, 6.0]

I = [0.00, 0.89, 1.65, 2.31, 2.91, 3.52, 4.18, 4.92, 5.60, 6.02,
     6.09, 5.91, 5.70, 5.59, 5.58, 5.64, 5.71, 5.78, 5.84, 5.89,
     5.93, 5.95, 5.98, 5.99, 6.00, 6.01, 6.00, 5.99, 5.98, 5.97,
     5.95, 5.93, 5.91, 5.88, 5.86, 5.83]

#### Part a, Plot the data and estimate the wavelength of the emission peak. ####
plt.plot(lam,I, label = "Original Data", color = "blue")
peak=lam[I.index(max(I))]
print(f"Wavelength of emission peak: {peak} Angstrom")


#### Part b, Create a spline fit for the background ####
lam_bg=[]
I_bg = []

for i in range(len(lam)):
    if lam[i] < 3.1 or lam[i] > 4.3:
        lam_bg.append(lam[i])
        I_bg.append(I[i])

lam_bg=np.array(lam_bg)
```

```
I_bg=np.array(I_bg)

CS_bg=CubicSpline(lam_bg, I_bg)
background = CS_bg(lam)

#### Part c, Subtract the background to isolate the emission peak. ####
#### i. Fit this peak to a Gaussian, and estimate the peak. ####
#### ii. Compare this peak to your estimate from the original data plot. ####

I_peak = np.array(I) - background

def gauss(x,A,x1,sigma):
    return A*np.exp(-(x-x1)**2/2/sigma**2)

max_idx=np.argmax(I_peak)
guess_A =I_peak[max_idx]
guess_x1 = lam[max_idx]
guess_sigma = 0.1

initial_guess = [guess_A ,guess_x1, guess_sigma]
popt, pcov = curve_fit(gauss,lam, I_peak, p0=initial_guess)

lam_smooth = np.linspace(min(lam), max(lam), 100)
gauss_fit = gauss(lam_smooth, *popt)

print(f"Estimated Peak Center (Wavelength): {popt[1]:.4f} Angstrom")

plt.plot(lam,background, label = "Background", color = "orange")
plt.plot(lam, I_peak, label= "Isolated Peak", color = "green")
plt.plot(lam_smooth, gauss_fit, "r-", label = "Gaussian Fit")
plt.legend()
plt.show()
```

```
Wavelength of emission peak: 3.5 Angstrom
Estimated Peak Center (Wavelength): 3.3911 Angstrom
```

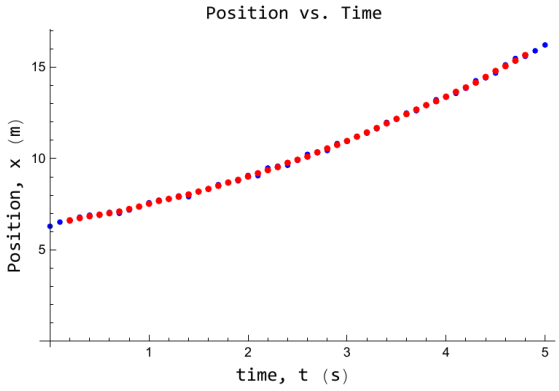


In [103]:
```
from pdf2image import convert_from_path
from IPython.display import display
images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W10
for img in images:
    display(img)
```

Problem 3.

In[ ]:=
```
t = {0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0, 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 1.7,
   1.8, 1.9, 2.0, 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9, 3.0, 3.1, 3.2, 3.3, 3.4,
   3.5, 3.6, 3.7, 3.8, 3.9, 4.0, 4.1, 4.2, 4.3, 4.4, 4.5, 4.6, 4.7, 4.8, 4.9, 5.0};
x = {6.29, 6.52, 6.58, 6.79, 6.9, 6.89, 7.05, 7.00, 7.18, 7.36, 7.58, 7.72, 7.78, 7.95, 7.91,
   8.18, 8.34, 8.57, 8.67, 8.78, 9.07, 9.06, 9.48, 9.58, 9.63, 9.9, 10.23, 10.32,
   10.44, 10.82, 10.95, 11.2, 11.39, 11.63, 11.98, 12.15, 12.49, 12.62, 12.93, 13.22,
   13.36, 13.57, 13.85, 14.26, 14.42, 14.68, 15.13, 15.48, 15.6, 15.9, 16.22};
data = Transpose[{t, x}];
smoothedData = MovingAverage[data, 5];
Dataplot = ListPlot[data, PlotStyle → Blue];
Labeled[Show[Dataplot, ListPlot[smoothedData, PlotStyle → Red ]],
 {"Position vs. Time", "time, t (s)", "Position, x (m)"},
 {Top, Bottom, Left}, RotateLabel → True]
```

Out[ ]=



In[•]:=
```
v = Table[(x[[i + 2]] - x[[i]])/(2 dt), {i, 1, Length[x] - 2}];
dt = 0.1;
t2 = Table[t[[i]], {i, 2, Length[x] - 1}];
vdata = Thread[{t2, v}];
vfit = LinearModelFit[vdata, V, V];
vfitsmooth = LinearModelFit[vdatasmooth, V, V];
vfit["ParameterTable"]
vfitplot = Plot[vfit[X], {X, Min[t], Max[t]}, PlotStyle → Black];
vplot = ListPlot[vdata, PlotStyle → Green];

xsmooth = MovingAverage[x, 5];
tsmooth = MovingAverage[t, 5];
vsmooth = Table[(xsmooth[[i + 2]] - xsmooth[[i]])/(2 dt), {i, 1, Length[xsmooth] - 2}];
tsmooth = Table[tsmooth[[i]], {i, 2, Length[xsmooth] - 1}];
vdatasmooth = Thread[{tsmooth, vsmooth}];
vfitsmooth = LinearModelFit[vdatasmooth, V, V];
vfitsmooth["ParameterTable"]
vfitplotsmooth = Plot[vfitsmooth[X], {X, Min[tsmooth], Max[tsmooth]}, PlotStyle → Red];
vplotsmooth = ListPlot[vdatasmooth, PlotStyle → Blue];

Labeled[Show[vfitplot, vplot, vfitplotsmooth, vplotsmooth ],
 {"Velocity vs. Time", "time, t (s)", "velocity, v (m/s)"},
 {Top, Bottom, Left}, RotateLabel → True]

vds = MovingAverage[v, 5];
tds = MovingAverage[t2, 5];
dt = 0.1;
vdatads = Thread[{tds, vds}];
vfitds = LinearModelFit[vdatads, V, V];
vfitds["ParameterTable"]
vfitplotds = Plot[vfitds[X], {X, Min[tds], Max[tds]}, PlotStyle → Red];
Labeled[vplotds = ListPlot[vdatads, PlotStyle → Blue],
  {"Velocity vs. Time", "time, t (s)", "velocity, v (m/s)"},
  {Top, Bottom, Left}, RotateLabel → True];
Labeled[Show[vfitplot, vplot, vfitplotds, vplotds],
 {"Velocity vs. Time", "time, t (s)", "velocity, v (m/s)"},
 {Top, Bottom, Left}, RotateLabel → True]
```

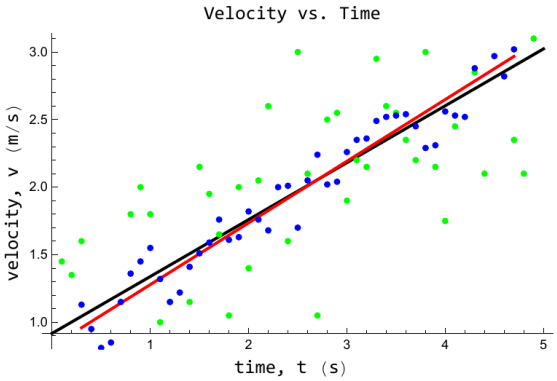
Out[•]=

	Estimate	Standard Error	t-Statistic	P-Value
1	0.917347	0.164819	5.56579	1.21457×10^{-6}
V	0.421224	0.0573826	7.34064	2.49296×10^{-9}

Out[•]=

	Estimate	Standard Error	t-Statistic	P-Value
1	0.822218	0.0515139	15.9611	1.11294×10^{-19}
V	0.456535	0.0182854	24.9672	3.20886×10^{-27}

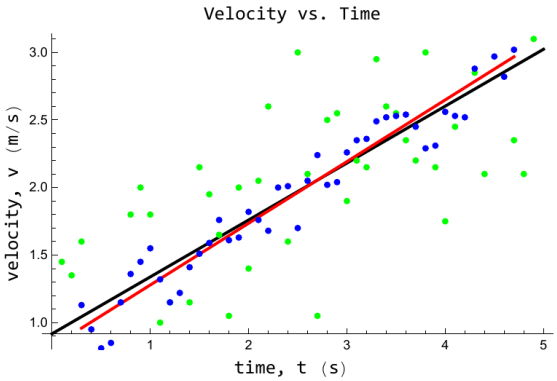
Out[•]=



Out[•]=

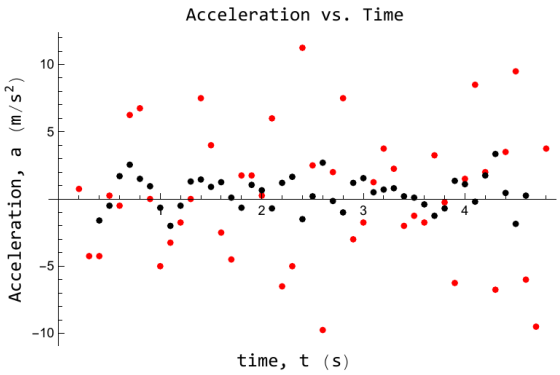
	Estimate	Standard Error	t-Statistic	P-Value
1	0.822218	0.0515139	15.9611	1.11294×10^{-19}
V	0.456535	0.0182854	24.9672	3.20886×10^{-27}

Out[•]=



In[•]:=
```
a = Table[(v[[i + 2]] - v[[i]])/(2 dt), {i, 1, Length[v] - 2}];
t3 = Table[t2[[i]], {i, 2, Length[v] - 1}];
adata = Thread[{t3, a}];
aplot = ListPlot[adata, PlotStyle → Red];

as = Table[(vsmooth[[i + 2]] - vsmooth[[i]])/(2 dt), {i, 1, Length[vsmooth] - 2}];
ts = Table[tsmooth[[i]], {i, 2, Length[vsmooth] - 1}];
adatas = Thread[{ts, as}];
Labeled[Show[aplot, aplots = ListPlot[adatas, PlotStyle → Black]],
 {"Acceleration vs. Time", "time, t (s)", "Acceleration, a (m/s^2)"},
 {Top, Bottom, Left}, RotateLabel → True]
Print[" Mean Accelaration: ", Mean[a],
  " | STD: ", StandardDeviation[a] ,
 " | Smoothed Mean Accelaration: ", Mean[as],
  " | Smoothed STD: ", StandardDeviation[as]]
```

Out[•]=



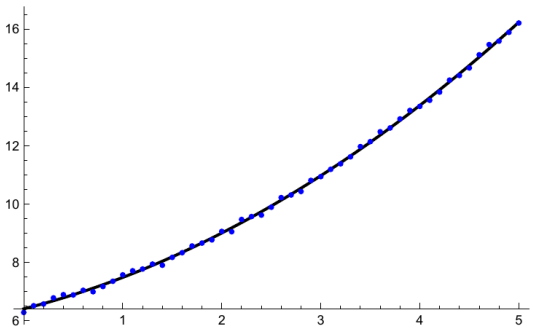

```
Mean Accelaration: 0.255319 | STD: 4.91545
 | Smoothed Mean Accelaration: 0.437209 | Smoothed STD: 1.23168
```

In[•]:=
```
xfit = NonlinearModelFit[data, a0 + a1 X + 1/2 a2 X^2, {a0, a1, a2}, X];
xfit["ParameterTable"]
xfitplot = Plot[xfit[X], {X, Min[t], Max[t]}, PlotStyle → Black];
Show[xfitplot, Dataplot]
```

Out[•]=

	Estimate	Standard Error	t-Statistic	P-Value
a0	6.41418	0.0302922	211.743	5.74196×10^{-73}
a1	0.855848	0.0280186	30.5457	4.15741×10^{-33}
a2	0.444292	0.0108384	40.9923	5.14572×10^{-39}

Out[•]=

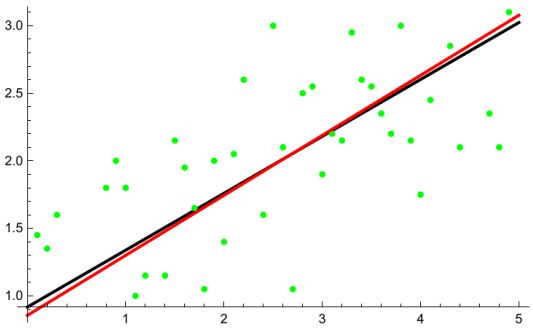



```
vfit = LinearModelFit[vdata, V, V];
vfit["ParameterTable"]
vfitplot = Plot[vfit[X], {X, Min[t], Max[t]}, PlotStyle → Black];
dvplot = Plot[(D[xfit[X], X] /. X → T), {T, Min[t], Max[t]}, PlotStyle → Red];
Show[vfitplot, vplot, dvplot]
```

Out[•]=

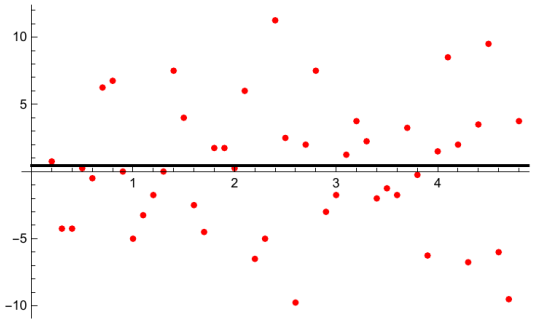
	Estimate	Standard Error	t-Statistic	P-Value
1	0.917347	0.164819	5.56579	1.21457×10^{-6}
V	0.421224	0.0573826	7.34064	2.49296×10^{-9}

Out[•]=



In[•]:=
```
daplot = Plot[D[xfit[X], {X, 2}] /. X → T, {T, Min[t], Max[t]}, PlotStyle → Black];
Show[aplot, daplot]
```

Out[•]=

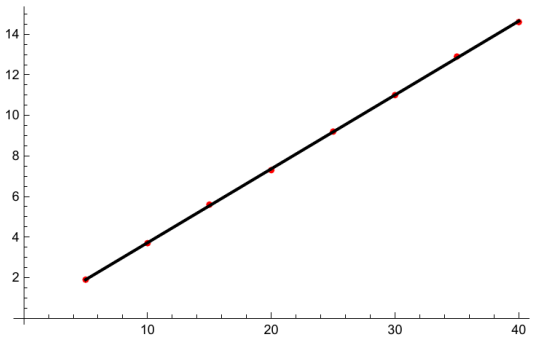


Problem 4.

In[58]:=
```
x = {5.0, 10.0, 15.0, 20.0, 25.0, 30.0, 35.0, 40.0};
F = {1.9, 3.7, 5.6, 7.3, 9.2, 11.0, 12.9, 14.6};
data = Thread[{x, F}];
lmf = LinearModelFit[data, X, X];
k = lmf["BestFitParameters"][[2]];
Print["Spring Constant k = ", k, " N/cm"];
dataplot = ListPlot[data, PlotStyle → Red];
fitplot = Plot[lmf[X], {X, Min[x], Max[x]}, PlotStyle → Black];
Show[dataplot, fitplot]
```

```
Spring Constant k = 0.364286 N/cm
```

Out[66]=



In[67]:=
```
rectangles = Table[x[[i]] (F[[i + 1]] - F[[i]]), {i, 1, Length[F] - 1}];
(* Create a rectangular approximation to the integral*)
intrect = Total[rectangles]
```

Out[68]=
```
253.
```

In[77]:=
```
trapezoids = Table[(x[[i]] + x[[i + 1]]) / 2 (F[[i + 1]] - F[[i]]), {i, 1, Length[F] - 1}];
(* Create a trapezoidal approximation to the integral*)
inttrap = Total[trapezoids]
```

Out[78]=
```
284.75
```

In[98]:=
$$F = \frac{1}{2} k * (\text{Max}[x])^2$$

Out[98]=
```
291.429
```

In[99]:=
```
int = Integrate[lmf[X], {X, Min[x], Max[x]}]
```

Out[99]=
```
289.625
```

In[100]:=
```
(* Comparing to results from the known function *)

Print[Column[{"Percent Difference:",
   Row[{"Rectangular Integration: ", Abs[intrect - F] / F * 100, "%"}],
   Row[{"Trapezoidal Integration: ", Abs[inttrap - F] / F * 100, "%"}],
   Row[{"Linear Model Fit: ", Abs[int - F] / F * 100, "%"}]}, Spacings → 1]]
```

```
Percent Difference:
Rectangular Integration: 13.1863%
Trapezoidal Integration: 2.29167%
Linear Model Fit: 0.618873%
```


In [ ]: